Fetch - Data Analyst Take Home (SQL in Python)

April 20, 2025

1 Jami McMillen

1.1 Exercise 1

[31]:
```
import pandas as pd
import matplotlib.pyplot as plt
import sqlite3
from IPython.display import display, HTML
from matplotlib.patches import Patch
```

Data Import

[18]:
```
# Load the CSV files provided for analysis
users = pd.read_csv("C:/Users/jamim/OneDrive/Desktop/Job Documents/Fetch/
 ↪USER_TAKEHOME.csv")
transactions = pd.read_csv("C:/Users/jamim/OneDrive/Desktop/Job Documents/Fetch/
 ↪TRANSACTION_TAKEHOME.csv")
products = pd.read_csv("C:/Users/jamim/OneDrive/Desktop/Job Documents/Fetch/
 ↪PRODUCTS_TAKEHOME.csv")
```

Data Review User Information

[19]:
```
# Show the user data
html_output = users.head(500).to_html(
    border=0,
    classes="table table-striped",
    escape=False
)
scrollable_html = f"""
<div style="overflow:auto; max-height:400px; width:100%; font-family:monospace;␣
 ↪white-space:nowrap;">
{html_output}
</div>
"""
display(HTML(scrollable_html))
```

```
<IPython.core.display.HTML object>
```

Notes: numerous nulls in language, several missing birth dates, dates are timestamps

Stats User

[20]:
```
# Check for missing data
fields = ['BIRTH_DATE', 'STATE', 'LANGUAGE', 'GENDER', 'CREATED_DATE']
total_rows = len(users)
missing_data = users[fields].isnull().sum() / total_rows * 100

print("Missing Data (%):")
print(missing_data.round(2))

# Confirm date
date_fields = ['BIRTH_DATE', 'CREATED_DATE']
parsed_status = {col: pd.api.types.is_datetime64_any_dtype(users[col]) for col
 ↪in date_fields}

print("\nDatetime Parsing Check:")
for col, is_parsed in parsed_status.items():
    print(f"{col}: {'parsed as datetime' if is_parsed else 'NOT parsed'}")
```

```
Missing Data (%):
BIRTH_DATE       3.68
STATE            4.81
LANGUAGE        30.51
GENDER           5.89
CREATED_DATE     0.00
dtype: float64

Datetime Parsing Check:
BIRTH_DATE: NOT parsed
CREATED_DATE: NOT parsed
```

Transations Information

[21]:
```
# Show the transaction data
html_output = transactions.head(500).to_html(
    border=0,
    classes="table table-striped",
    escape=False
)
scrollable_html = f"""
<div style="overflow:auto; max-height:400px; width:100%; font-family:monospace;
 ↪white-space:nowrap;">
{html_output}
</div>
"""
display(HTML(scrollable_html))
```

```
<IPython.core.display.HTML object>
```

[22]:
```
# Summary stats
summary_stats = transactions[['FINAL_QUANTITY', 'FINAL_SALE']].describe()
print(summary_stats.round(2))
```

```
       FINAL_QUANTITY FINAL_SALE
count           50000      50000
unique             87       1435
top              1.00
freq            35698      12500
```

Products Information

[23]:
```
# More sum stats

total = len(products)

# Clean column names just in case
products.columns = products.columns.str.strip()

# Missing %
barcode_missing = products['BARCODE'].isnull().sum() / total * 100
cat_missing = products[['CATEGORY_1', 'CATEGORY_2', 'CATEGORY_3', 'CATEGORY_4',␣
 ↪'MANUFACTURER', 'BRAND']].isnull().sum() / total * 100

print(f"{total:,} product records across {products.shape[1]} fields\n")
print("Missing Data:")
print(f"BARCODE missing: {barcode_missing:.2f}%")
for col, pct in cat_missing.items():
    print(f"{col} missing: {pct:.2f}%")
```

```
845,552 product records across 7 fields

Missing Data:
BARCODE missing: 0.48%
CATEGORY_1 missing: 0.01%
CATEGORY_2 missing: 0.17%
CATEGORY_3 missing: 7.16%
CATEGORY_4 missing: 92.02%
MANUFACTURER missing: 26.78%
BRAND missing: 26.78%
```

Notes: different date formats, barcode in scientific notation format, final sale has blanks not NaN values,

[24]:
```
# Show the transaction data
html_output = products.head(500).to_html(
    border=0,
    classes="table table-striped",
    escape=False
```

```
)
scrollable_html = f"""
<div style="overflow:auto; max-height:400px; width:100%; font-family:monospace;␣
 ↪white-space:nowrap;">
{html_output}
</div>
"""
display(HTML(scrollable_html))
```

```
<IPython.core.display.HTML object>
```

Notes: barcode in scientific notation format, catorgy is a in a hjierarchical format

[25]:
```
# Summary stats for categorical columns in 'products'
cols = ['CATEGORY_1', 'CATEGORY_2', 'CATEGORY_3', 'CATEGORY_4', 'MANUFACTURER',␣
 ↪'BRAND']
df = products[cols]

summary = pd.DataFrame(index=cols)
summary['missing'] = df.isnull().sum()
summary['non_null'] = df.notnull().sum()
summary['unique'] = df.nunique()
summary['top'] = df.mode().iloc[0]
summary['top_freq'] = df.apply(lambda x: x.value_counts(dropna=False).iloc[0])

print(summary)
```

```
              missing  non_null  unique                       top  top_freq
CATEGORY_1        111    845441      27         Health & Wellness    512695
CATEGORY_2       1424    844128     121                     Candy    121036
CATEGORY_3      60566    784986     344          Confection Candy     60566
CATEGORY_4     778093     67459     127                 Lip Balms    778093
MANUFACTURER   226474    619078    4354  PLACEHOLDER MANUFACTURER    226474
BRAND          226472    619080    8122                 REM BRAND    226472
```

[26]:
```
products.columns = products.columns.str.strip()


# Missing % for key fields
missing = products[['CATEGORY_4', 'MANUFACTURER', 'BRAND', 'BARCODE']].isnull().
 ↪sum() / total * 100


cat1_counts = products['CATEGORY_1'].value_counts(normalize=True) * 100
top_cat1 = cat1_counts.head(5).round(2)
```

```python
placeholder_mfg = products['MANUFACTURER'].fillna('').str.
  ↪contains("PLACEHOLDER", case=False).sum()
null_brands = products['BRAND'].isnull().sum()

# Print output
print(f"{total:,} product records with {products.shape[1]} columns\n")

print("High Missingness:")
print(f"CATEGORY_4: {missing['CATEGORY_4']:.2f}%")
print(f"MANUFACTURER: {missing['MANUFACTURER']:.2f}%")
print(f"BRAND: {missing['BRAND']:.2f}%")
print(f"BARCODE: {missing['BARCODE']:.2f}%")

print("\nTop CATEGORY_1 values (as % of total):")
print(top_cat1)

print("\nTop CATEGORY_1 values (as % of total):")
print(top_cat1)

print(f"\nPlaceholder manufacturer values: {placeholder_mfg:,}")
print(f"Null brand values: {null_brands:,}")
```

```
845,552 product records with 7 columns

High Missingness:
CATEGORY_4: 92.02%
MANUFACTURER: 26.78%
BRAND: 26.78%
BARCODE: 0.48%

Top CATEGORY_1 values (as % of total):
CATEGORY_1
Health & Wellness        60.64
Snacks                   38.42
Beverages                 0.47
Pantry                    0.10
Apparel & Accessories     0.10
Name: proportion, dtype: float64

Top CATEGORY_1 values (as % of total):
CATEGORY_1
Health & Wellness        60.64
Snacks                   38.42
Beverages                 0.47
Pantry                    0.10
Apparel & Accessories     0.10
Name: proportion, dtype: float64
```

```
Placeholder manufacturer values: 86,902
Null brand values: 226,472
```

1.1.1 Comments – Data Exploration Summary

1.1.2 USERS TABLE

- 100,000 user records with 6 fields

- **Missing Data:**
  - `BIRTH_DATE`: **3.68%**
  - `STATE`: **4.81%**
  - `LANGUAGE`: **30.51%** ← *significant*
  - `GENDER`: **5.89%**
- Date fields (`BIRTH_DATE`, `CREATED_DATE`) were successfully parsed into datetime
- **Assumption**: Missing demographic fields will be excluded unless a justified imputation strategy is proposed

---

1.1.3 TRANSACTIONS TABLE

- 50,000 transaction records across 8 fields

- **Data Issues:**
  - `BARCODE` missing in **11.52%** → may impact joins
  - `FINAL_QUANTITY` and `FINAL_SALE` missing in **25%** of rows — many include `"zero"` or blank values
  - `FINAL_QUANTITY` values are not all whole numbers
- Cleaned `FINAL_QUANTITY` ranges from small decimals to whole numbers
- `FINAL_SALE` ranges from **$0.25** to **$462.82**, with a mean of ~**$4.58** — suggests possible outliers

---

1.1.4 PRODUCTS TABLE

- ~845,000 product records with 7 columns

- **High Missingness:**
  - `CATEGORY_4`: **92.02%** → not usable
  - `MANUFACTURER` and `BRAND`: ~**27%** missing — impacts brand-level analysis
- `BARCODE` missing in <1% (**0.48%**) — but it's critical for joins
- **Top `CATEGORY_1` values** are heavily skewed:
  - Health & Wellness, Snacks, Beverages dominate the catalog
- Placeholder values like `"PLACEHOLDER MANUFACTURER"` and `null` brands were observed — these will be excluded

---

1.1.5 OVERALL ASSUMPTIONS

- Records missing `FINAL_SALE`, `BARCODE`, or `BRAND` will be excluded from core analysis
- User age will be calculated from `BIRTH_DATE` assuming the current year is **2025**
- Users will be filtered by account age (6+ months) using `CREATED_DATE`
- Health & Wellness analysis may be skewed due to overrepresentation in product catalog

1.2 Exercise 2

[29]:
```
# Setup SQLite database
conn = sqlite3.connect(":memory:")


# Load data into SQLite tables
users.to_sql("Users", conn, index=False, if_exists="replace")
transactions.to_sql("Transactions", conn, index=False, if_exists="replace")
products.to_sql("Products", conn, index=False, if_exists="replace")


# Top 5 brands by receipts scanned (21+)
query_21_plus = """
WITH user_age AS (
  SELECT ID AS user_id,
         CAST(strftime('%Y', 'now') AS INTEGER) - CAST(strftime('%Y',␣
 ↪BIRTH_DATE) AS INTEGER) AS age
  FROM Users
  WHERE BIRTH_DATE IS NOT NULL
),
eligible_users AS (
  SELECT user_id FROM user_age WHERE age >= 21
)
SELECT
    p.BRAND,
    COUNT(DISTINCT t.RECEIPT_ID) AS receipt_count
FROM Transactions t
JOIN eligible_users u ON t.USER_ID = u.user_id
JOIN Products p ON t.BARCODE = p.BARCODE
WHERE p.BRAND IS NOT NULL
GROUP BY p.BRAND
ORDER BY receipt_count DESC
LIMIT 5;
"""


# Top 5 brands by sales (users with accounts 6+ months)
query_sales_6mo = """
WITH eligible_users AS (
  SELECT ID AS user_id
```

```
  FROM Users
  WHERE julianday('now') - julianday(CREATED_DATE) >= 180
)
SELECT
    p.BRAND,
    SUM(CAST(t.FINAL_SALE AS REAL)) AS total_sales
FROM Transactions t
JOIN eligible_users u ON t.USER_ID = u.user_id
JOIN Products p ON t.BARCODE = p.BARCODE
WHERE t.FINAL_SALE IS NOT NULL AND p.BRAND IS NOT NULL
GROUP BY p.BRAND
ORDER BY total_sales DESC
LIMIT 5;
"""


# % of sales in Health & Wellness by generation
query_health_by_gen = """
WITH user_gen AS (
  SELECT ID AS user_id,
         CASE
           WHEN CAST(strftime('%Y', BIRTH_DATE) AS INT) BETWEEN 1946 AND 1964␣
 ↪THEN 'Boomers'
           WHEN CAST(strftime('%Y', BIRTH_DATE) AS INT) BETWEEN 1965 AND 1980␣
 ↪THEN 'Gen X'
           WHEN CAST(strftime('%Y', BIRTH_DATE) AS INT) BETWEEN 1981 AND 1996␣
 ↪THEN 'Millennials'
           WHEN CAST(strftime('%Y', BIRTH_DATE) AS INT) >= 1997 THEN 'Gen Z'
           ELSE 'Other'
         END AS generation
  FROM Users
  WHERE BIRTH_DATE IS NOT NULL
),
joined AS (
  SELECT
    g.generation,
    p.CATEGORY_1,
    CAST(t.FINAL_SALE AS REAL) AS sale
  FROM Transactions t
  JOIN user_gen g ON t.USER_ID = g.user_id
  JOIN Products p ON t.BARCODE = p.BARCODE
  WHERE t.FINAL_SALE IS NOT NULL
)
SELECT
  generation,
  ROUND(100.0 * SUM(CASE WHEN CATEGORY_1 = 'Health & Wellness' THEN sale ELSE 0␣
 ↪END) / SUM(sale), 2) AS health_percentage
```

```
FROM joined
GROUP BY generation;
"""


# Open-ended: Who are Fetch's Power Users?
# Assumption: Power User -> Total Spend + Reciept Count
query_power_users = """
WITH user_sales AS (
  SELECT
    t.USER_ID,
    COUNT(DISTINCT t.RECEIPT_ID) AS total_receipts,
    SUM(CAST(t.FINAL_SALE AS REAL)) AS total_spend
  FROM Transactions t
  WHERE t.FINAL_SALE IS NOT NULL
  GROUP BY t.USER_ID
)
SELECT
    u.ID,
    CAST(strftime('%Y', 'now') AS INTEGER) - CAST(strftime('%Y', u.
 ↪CREATED_DATE) AS INTEGER) AS age_creadted_date,
    CAST(strftime('%Y', 'now') AS INTEGER) - CAST(strftime('%Y', u.BIRTH_DATE)␣
 ↪AS INTEGER) AS age,
    u.STATE,
    u.LANGUAGE,
    u.GENDER,
    us.total_receipts,
    us.total_spend
FROM user_sales us
JOIN Users u ON u.ID = us.USER_ID
WHERE us.total_receipts > 0 AND us.total_spend > 0
ORDER BY us.total_spend DESC, us.total_receipts DESC
LIMIT 20;
"""

# Open-ended: Leading brand in the Dips & Salsa category
query_dips_salsa = """
SELECT
    p.BRAND,
    SUM(CAST(t.FINAL_SALE AS REAL)) AS total_sales
FROM Transactions t
JOIN Products p ON t.BARCODE = p.BARCODE
WHERE p.CATEGORY_2 = 'Dips & Salsa'
  AND t.FINAL_SALE IS NOT NULL
  AND p.BRAND IS NOT NULL
GROUP BY p.BRAND
ORDER BY total_sales DESC
```

```
LIMIT 5;
"""

#Year-ove r-year Fetch growth
query_growth_yoy = """
WITH user_years AS (
  SELECT
    strftime('%Y', CREATED_DATE) AS year,
    COUNT(*) AS user_count
  FROM Users
  WHERE CREATED_DATE IS NOT NULL
  GROUP BY year
),
growth_calc AS (
  SELECT
    year,
    user_count,
    LAG(user_count) OVER (ORDER BY year) AS prev_year_count
  FROM user_years
)
SELECT
  year,
  user_count,
  prev_year_count,
  ROUND(100.0 * (user_count - prev_year_count) / prev_year_count, 2) AS␣
 ↪growth_percent
FROM growth_calc
WHERE prev_year_count IS NOT NULL;
"""

# Run each query
result1 = pd.read_sql_query(query_21_plus, conn)
result2 = pd.read_sql_query(query_sales_6mo, conn)
result3 = pd.read_sql_query(query_health_by_gen, conn)
result4 = pd.read_sql_query(query_power_users, conn)
result5 = pd.read_sql_query(query_dips_salsa, conn)
result6 = pd.read_sql_query(query_growth_yoy, conn)

# Show results
print("Top 5 Brands by Receipts (21+ Users):")
print(result1, "\n")

print("Top 5 Brands by Sales (Users with 6+ Months Accounts):")
print(result2, "\n")

print("% of Health & Wellness Sales by Generation:")
print(result3, "\n")
```

```
print("Top 5 Power Users (receipt count + highest spend):")
print(result4, "\n")

print("Leading brand in the Dips & Salsa category:")
print(result5, "\n")

print("Year-over-year Fetch growth:")
print(result6, "\n")
```

```
Top 5 Brands by Receipts (21+ Users):
             BRAND  receipt_count
0      NERDS CANDY              3
1             DOVE              3
2          TRIDENT              2
3  SOUR PATCH KIDS              2
4           MEIJER              2

Top 5 Brands by Sales (Users with 6+ Months Accounts):
         BRAND  total_sales
0          CVS        72.00
1      TRIDENT        46.72
2         DOVE        42.88
3  COORS LIGHT        34.96
4       QUAKER        16.60

% of Health & Wellness Sales by Generation:
    generation  health_percentage
0      Boomers              39.47
1        Gen X              29.87
2        Gen Z               0.00
3  Millennials              39.96
4        Other               0.00

Top 5 Power Users (receipt count + highest spend):
                          ID  age_creadted_date  age STATE LANGUAGE  GENDER  \
0   643059f0838dd2651fb27f50                  2   71    PA       en    male
1   62ffec490d9dbaff18c0a999                  3   74    NY       en  female
2   5f4c9055e81e6f162e3f6fa8                  5   38    FL   es-419  female
3   5d191765c8b1ba28e74e8463                  6   61    TX       en  female
4   6351760a3a4a3534d9393ecd                  3   41    FL       en  female
5   64dd9170516348066e7c4006                  2   32    SC       en  female
6   62c09104baa38d1a1f6c260e                  3   47    PA       en  female
7   61a58ac49c135b462ccddd1c                  4   29    TN       en  female
8   6661ed1e7c0469953bfc76c4                  1   34    FL       en  female
9   5b441360be53340f289b0795                  7   43    IL       en  female
10  61730bba65abe727fff3fcf7                  4   71    MO       en  female
11  5f21e60446f11314a16015de                  5   45    NC       en    male
```

```
12  610a8541ca1fab5b417b5d33                  4   48    NY       en    male
13  5c366bf06d9819129dfa1118                  6   36    NC       en  female
14  6682b24786cc41b000ce5e77                  1   37    WI       en  female
15  632fc9dc0c625b72ae991f83                  3   45    CA   es-419  female
16  646bdaa67a342372c857b958                  2   44    WI       en  female
17  63c2d13139c79dcbdd563f61                  2   36    VA       en    male
18  5ea899202244e629eacd6785                  5   50    DE       en  female
19  5fc12a8a16770448f92e56b8                  5   38    TX       en  female
```

	total_receipts	total_spend
0	2	75.99
1	3	52.28
2	1	37.96
3	1	34.96
4	2	27.74
5	2	26.52
6	3	20.28
7	3	19.92
8	1	18.60
9	2	18.32
10	1	18.22
11	2	17.96
12	3	17.65
13	3	17.42
14	2	17.30
15	2	16.75
16	2	15.74
17	2	15.56
18	1	15.12
19	2	14.67

Leading brand in the Dips & Salsa category:

	BRAND	total_sales
0	TOSTITOS	260.99
1	GOOD FOODS	118.89
2	PACE	118.58
3	MARKETSIDE	103.29
4	FRITOS	91.73

Year-over-year Fetch growth:

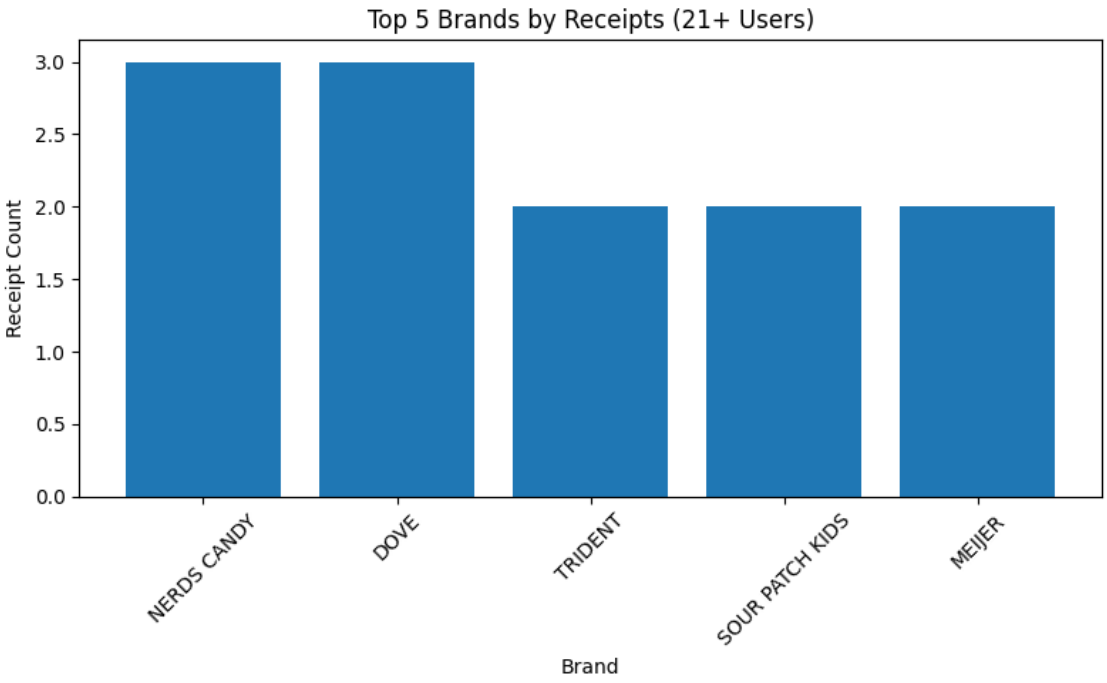
	year	user_count	prev_year_count	growth_percent
0	2015	51	30	70.00
1	2016	70	51	37.25
2	2017	644	70	820.00
3	2018	2168	644	236.65
4	2019	7093	2168	227.17
5	2020	16883	7093	138.02
6	2021	19159	16883	13.48

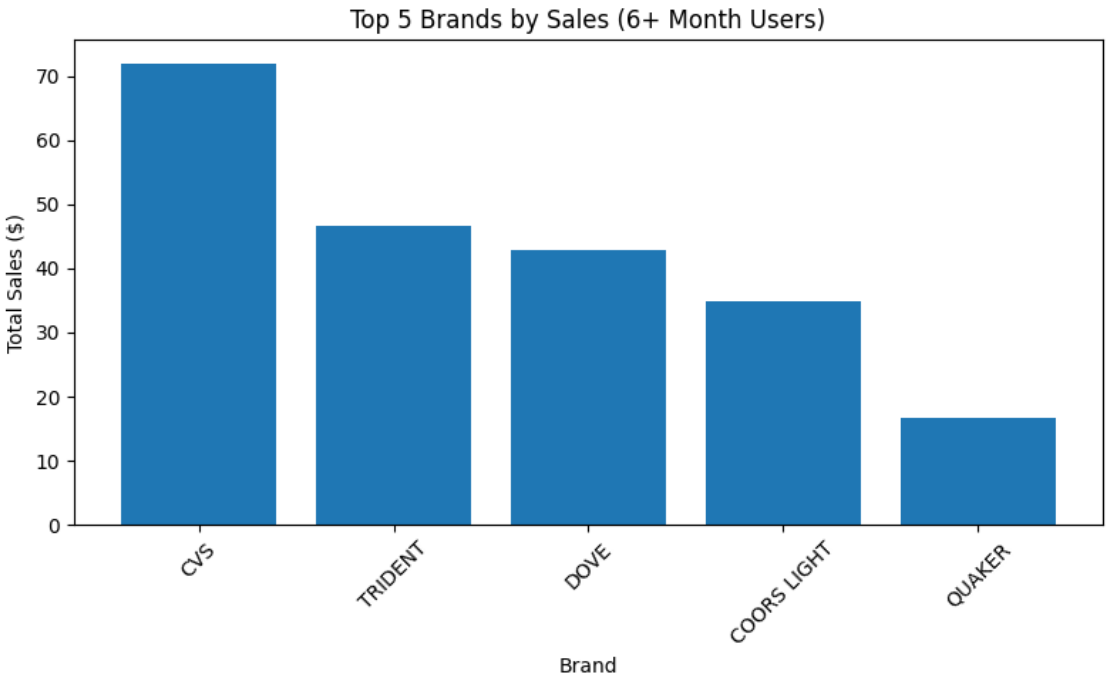
```
7  2022        26807             19159            39.92
8  2023        15464             26807           -42.31
9  2024        11631             15464           -24.79
```

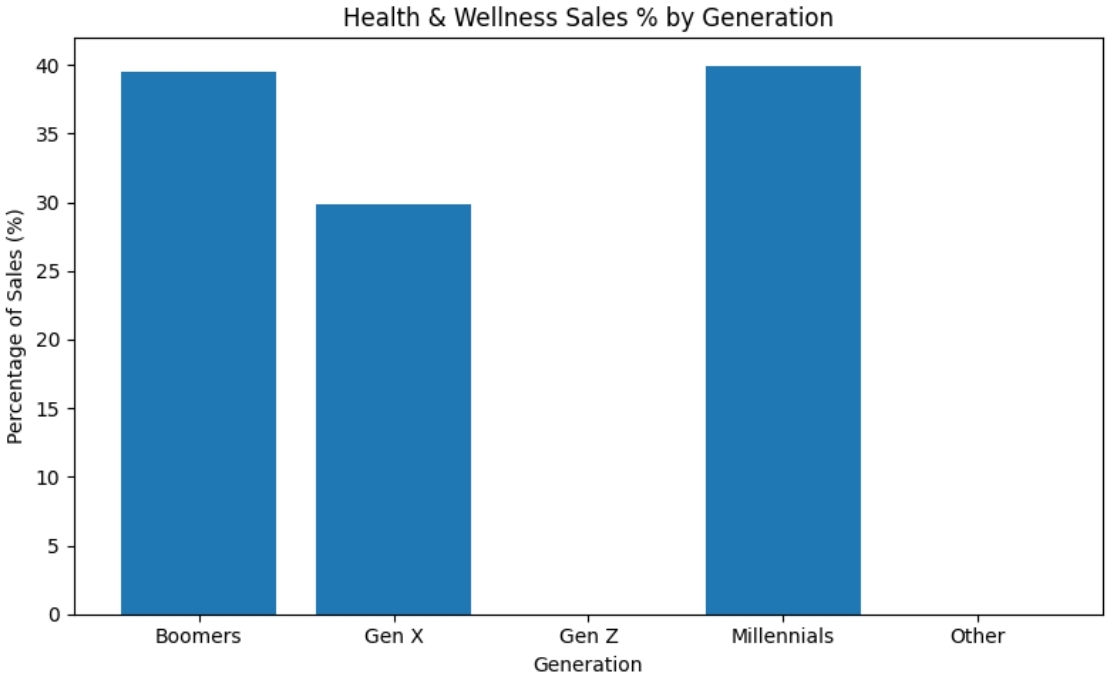
[32]:
```python
import matplotlib.pyplot as plt

# 1. Bar chart for Top 5 brands by receipts scanned (21+)
plt.figure(figsize=(8, 5))
plt.bar(result1['BRAND'], result1['receipt_count'])
plt.title('Top 5 Brands by Receipts (21+ Users)')
plt.xlabel('Brand')
plt.ylabel('Receipt Count')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 2. Bar chart for Top 5 brands by sales (6+ month users)
plt.figure(figsize=(8, 5))
plt.bar(result2['BRAND'], result2['total_sales'])
plt.title('Top 5 Brands by Sales (6+ Month Users)')
plt.xlabel('Brand')
plt.ylabel('Total Sales ($)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 3. Bar chart for % Health & Wellness sales by generation
plt.figure(figsize=(8, 5))
plt.bar(result3['generation'], result3['health_percentage'])
plt.title('Health & Wellness Sales % by Generation')
plt.xlabel('Generation')
plt.ylabel('Percentage of Sales (%)')
plt.tight_layout()
plt.show()

# 4 Top Power Users

user_ids = result4['ID'].astype(str).str[:10]
total_receipts = result4['total_receipts']
total_spend = result4['total_spend']
rank_labels = [f"Top {i+1}" for i in range(len(result4))]

# labels
legend_entries = []
for i in range(len(result4)):
```

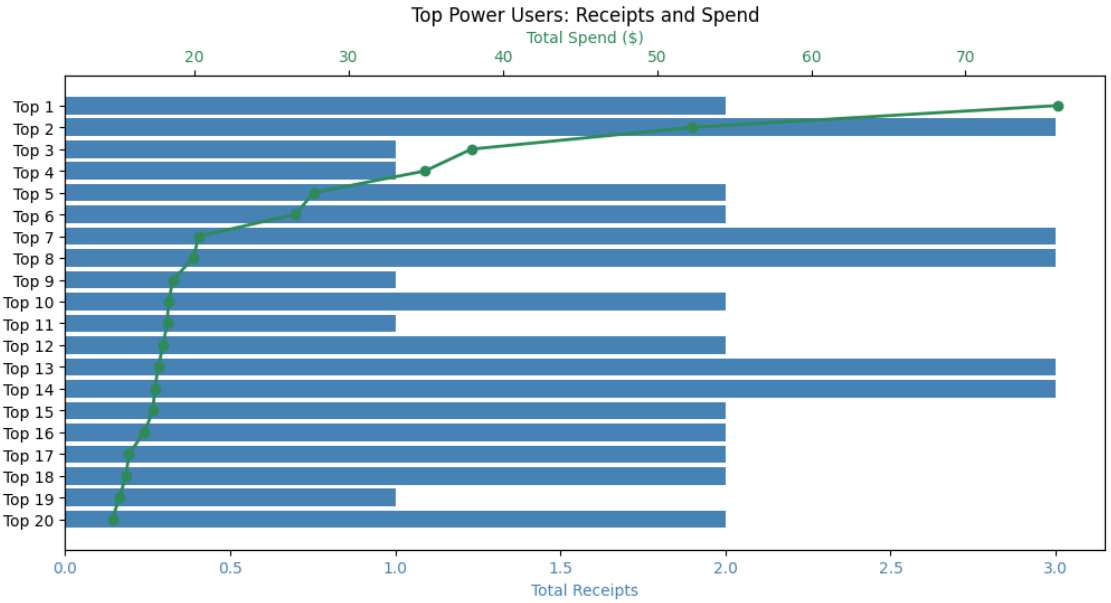
```
    label = f"Top {i+1}: {user_ids.iloc[i]} - {total_receipts.iloc[i]}␣
 ↪receipts, ${total_spend.iloc[i]:,.2f}"
    legend_entries.append(Patch(color='white', label=label))

fig, ax1 = plt.subplots(figsize=(12, 8))

# otal Receipts
bars = ax1.barh(rank_labels, total_receipts, color='steelblue')
ax1.set_xlabel('Total Receipts', color='steelblue')
ax1.tick_params(axis='x', labelcolor='steelblue')
ax1.set_title('Top Power Users: Receipts and Spend')
ax1.invert_yaxis()

# Total Spend
ax2 = ax1.twiny()
ax2.plot(total_spend, rank_labels, color='seagreen', marker='o', linewidth=2)
ax2.set_xlabel('Total Spend ($)', color='seagreen')
ax2.tick_params(axis='x', labelcolor='seagreen')

# Add legend
plt.legend(
    handles=legend_entries,
    title='User Mapping',
    loc='upper center',
    bbox_to_anchor=(0.5, -0.25),
    frameon=False,
    fontsize=9,
    title_fontsize=10
)

plt.subplots_adjust(bottom=0.35)
plt.show()


# 5. Bar chart for Top brands in Dips & Salsa
plt.figure(figsize=(8, 5))
plt.bar(result5['BRAND'], result5['total_sales'])
plt.title('Top Brands in Dips & Salsa')
plt.xlabel('Brand')
plt.ylabel('Total Sales ($)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 6. Line chart for Year-over-Year Growth %
plt.figure(figsize=(10, 5))
plt.plot(result6['year'], result6['growth_percent'], marker='o', linestyle='-')
```

```
plt.title('Year-over-Year User Growth %')
plt.xlabel('Year')
plt.ylabel('Growth %')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



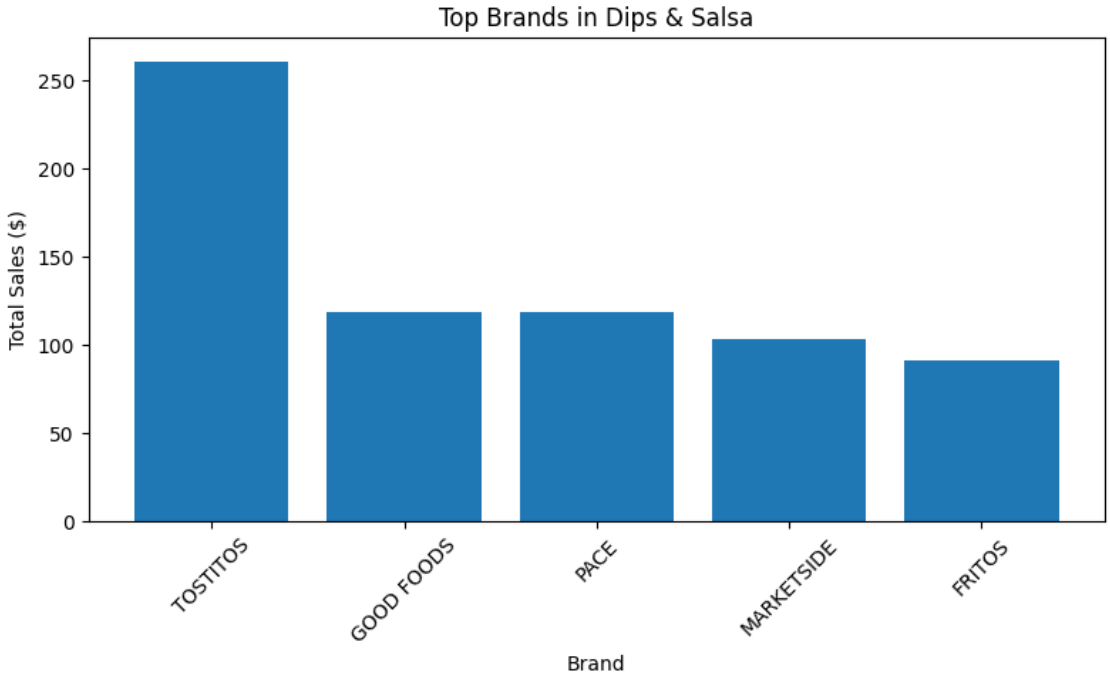

Top 5 Brands by Sales (6+ Month Users)
Total Sales ($)
0
10
20
30
40
50
60
70
CVS
TRIDENT
DOVE
COORS LIGHT
QUAKER
Brand

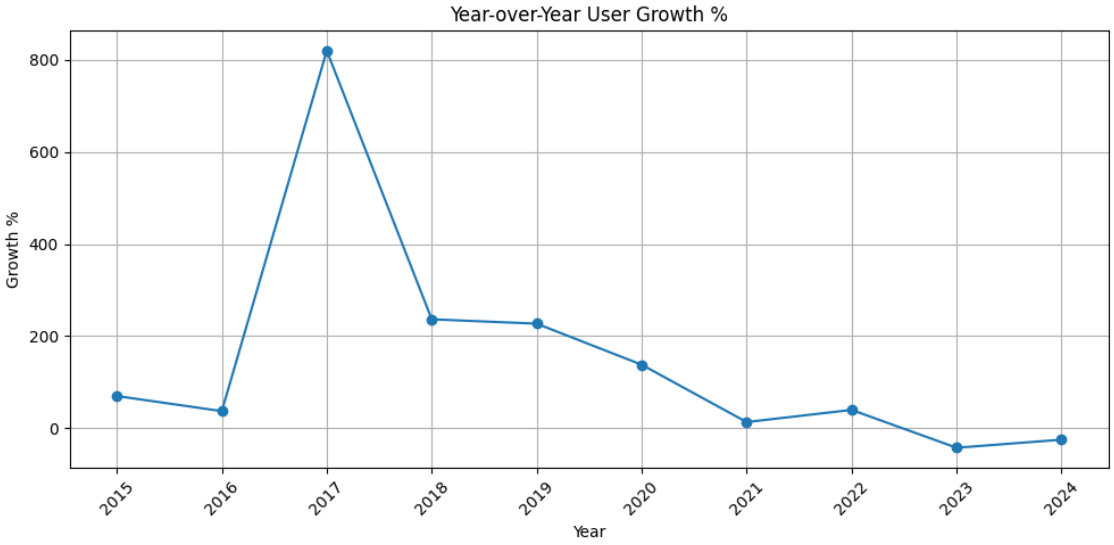

Health & Wellness Sales % by Generation
Percentage of Sales (%)
0
5
10
15
20
25
30
35
40
Boomers
Gen X
Gen Z
Millennials
Other
Generation



User Mapping

Top 1: 643059f083 — 2 receipts, $75.99
Top 2: 62ffec490d — 3 receipts, $52.28
Top 3: 5f4c9055e8 — 1 receipts, $37.96
Top 4: 5d191765c8 — 1 receipts, $34.96
Top 5: 6351760a3a — 2 receipts, $27.74
Top 6: 64dd917051 — 2 receipts, $26.52
Top 7: 62c09104ba — 3 receipts, $20.28
Top 8: 61a58ac49c — 3 receipts, $19.92
Top 9: 6661ed1e7c — 1 receipts, $18.60
Top 10: 5b441360be — 2 receipts, $18.32
Top 11: 61730bba65 — 1 receipts, $18.22
Top 12: 5f21e60446 — 2 receipts, $17.96
Top 13: 610a8541ca — 3 receipts, $17.65
Top 14: 5c366bf06d — 3 receipts, $17.42
Top 15: 6682b24786 — 2 receipts, $17.30
Top 16: 632fc9dc0c — 2 receipts, $16.75
Top 17: 646bdaa67a — 2 receipts, $15.74
Top 18: 63c2d13139 — 2 receipts, $15.56
Top 19: 5ea8992022 — 1 receipts, $15.12
Top 20: 5fc12a8a16 — 2 receipts, $14.67





1.3 Exercise 3

1.3.1 Fetch Data Analysis Summary

Hi (product or business leader),

Please review the initial data analysis across the **Fetch** user, transaction, and product datasets.

Below is a summary of findings, trends, and next steps:

1.3.2 Key Data Quality Issues

- **Users**
  - `LANGUAGE`: missing in **30.5%**
  - `GENDER`: missing in **5.9%**
  - `BIRTH_DATE`: missing in **3.7%**
    → These gaps limit demographic segmentation.
- **Transactions**
  - `FINAL_QUANTITY` and `FINAL_SALE`: **25%** have invalid values (e.g., `"zero"`, blanks)
  - `BARCODE`: missing in **11.5%**, limiting product joins
- **Products**
  - `CATEGORY_4`: missing in **92%** of records
  - `BRAND` and `MANUFACTURER`: missing in ~**27%**
    → Impacts brand-level insights and joins

1.3.3 Key Insights

- Users **21+** most frequently scanned receipts for:
  NERDS CANDY, **DOVE**, and **TRIDENT**
- Long-term users (**6+ months**) contributed the most sales to:
  CVS, **TRIDENT**, and **DOVE**
- **Health & Wellness** spending is driven by:
  - **Boomers** (39.5%)
  - **Millennials** (40%)
  - **Gen Z** contributes **0%** — potentially a capture issue
- **TOSTITOS** leads the **Dips & Salsa** category
  - Followed by **GOOD FOODS**, **PACE**, and **MARKETSIDE**
- **Year-over-Year Growth**
  - Peaked at **820%** in **2017**
  - Declined by **-42.3%** in **2023** and **-24.8%** in **2024**

1.3.4 Request for Action

- Should we exclude rows with `"PLACEHOLDER MANUFACTURER"` or missing brand fields?
- Should empty `FINAL_SALE` be treated as **$0** or **unrecorded**?

- Can we confirm that excluding rows missing `BARCODE` or `BRAND` is acceptable?

Please let me know if you have questions or feedback.

For next steps, we can explore dashboard visualizations or deeper dives into user segments, product trends, or brand-switching behavior.

Thanks,
Jami McMillen